REFERENCE GUIDE

Power Glove Vision Installation Guide

PowerGlove Vision documentation

2026 EDITION / IAIN BENNETT / MIT LICENSE

Follow this guide to download PowerGlove Vision, install it on both computers, pair them, and use your hand to control a game. At the end, RetroArch should recognize a virtual controller named **PowerGlove Vision**.

This guide describes the current `dev` version. Use the same branch on both machines. A future release may have different steps; use its matching guide. The setup commands have been checked against the code and an existing cabinet. Installation on a completely fresh system has not yet been verified.

Before you start

You need a working RetroPie system with RetroArch, an Arduino UNO Q, a UVC-compatible USB camera, a powered USB hub, and a computer running Arduino App Lab. The UNO Q and RetroPie must share a trusted local network. Internet access is needed to download the software and prepare the vision runtime. Keep a physical controller available for configuring RetroArch and recovering from an incorrect gesture mapping. Supply your own legally obtained games; PowerGlove Vision does not include ROMs or BIOS files.

Machine	What you do there
Development computer	Download the source, build the App Lab ZIP, and import it into the UNO Q
UNO Q	Run the camera, web interface, and gesture tracker
Raspberry Pi running RetroPie	Install the receiver and connect the virtual gamepad to RetroArch

Replace `UNO-Q-NAME.local` and `RETROPIE-NAME.local` with your devices' actual hostnames. A hostname is an address such as `myconsole.local`; it does not include `http://`, a port, or a page path. The commands below use a macOS or Linux terminal. Run them one line at a time and stop if a command reports an error. A command block contains only text to enter; do not type the fence marks.

All project flags and the system-command options used here are explained in the command reference. Returning users can go directly to the Play Checklist.

Stage 1 - Download the software on your computer

1. On your development computer, install Arduino App Lab and open a terminal.
2. Check that `git`, `python3`, `bash`, `rsync`, and `zip` are available by running `command -v git python3 bash rsync zip`. Each name should produce a path. If a tool is missing, install it using your operating system's package manager or the tool's official installer before continuing. The packaging script requires macOS or a Linux shell; Windows users need a compatible Linux environment for this step.
3. Choose a folder for your project checkout. Run the download commands below from that folder.

```
SH
git clone --branch dev https://github.com/mathan416/PowerGlove-Vision.git
cd PowerGlove-Vision
git branch --show-current
```

The last command should print `dev`. The downloaded folder contains `README.md`, `src/`, `scripts/`, `docs/`, and the other project files. If the folder already exists, use your existing checkout; do not clone over it or discard local changes.

Build the UNO Q installation ZIP

1. In the terminal on your development computer, stay in the `PowerGlove-Vision` folder.
2. Run the build and verification commands below.
3. Confirm that verification reports **App Lab installation ZIP verified**. The ZIP is ready to import; do not unzip it first.

```
SH
scripts/build-app-lab-package.sh
python3 scripts/verify-app-lab-package.py
```

The resulting file is `output/app-lab/PowerGlove-Vision-Uno-Q.zip` inside your checkout. It contains the application, matrix sketch, public guides, and MediaPipe wheel, verified hand-tracking model, Apache 2.0 license, and third-party notices. It excludes private settings. The UNO Q verifies and installs the bundled model when first needed.

Stage 2 - Install the application on the UNO Q

Connect and import

1. Connect the UNO Q to your development computer with a USB data cable and open Arduino App Lab.
2. Complete the board setup, record its hostname, and connect it to the same trusted network as RetroPie. Use Arduino's App Lab documentation if the board has not been configured before.
3. In App Lab, import the ZIP built in Stage 1. This transfers the application and sketch to the UNO Q; downloading the Git repository on your computer alone does not install them on the board.
4. Open the imported **PowerGlove Vision** application and select **Run**. Allow the initial runtime preparation to finish. Later starts reuse the downloaded runtime.
5. Connect the camera to the UNO Q through the powered USB hub. Keep the hub powered, and use App Lab over the network when the hub occupies the UNO Q's USB connection.

Complete host setup

1. On your development computer, run `ssh arduino@UNO-Q-NAME.local`, using your board's actual hostname. Enter the UNO Q account password when prompted. The terminal now runs commands on the UNO Q.
2. Run the commands below on the UNO Q. The first command must succeed: this installer requires the exact application directory shown. If your import has a different directory, stop and resolve that App Lab installation path before running setup.
3. Read the installer report. Correct every **FAIL**. An **ACTION** asking you to pair or verify gameplay is expected at this stage.
4. Run `exit` to return to your development computer's terminal.

```
SH
cd /home/arduino/ArduinoApps/powerglove-vision
sudo python3 scripts/setup-machine.py uno-q
```

Setup installs local-name resolution and the shutdown helper, configures the secure web port, sets the application to start at boot, and restarts it. It preserves private settings. You do not need to install the shutdown helper a second time or copy

the matrix sketch separately.

Check the application

1. In a browser on your development computer, open `http://UNO-Q-NAME.local:8088/debug`.
2. Confirm that Dashboard loads. A camera error does not prevent the website from opening. With **Gestures off**, a closed camera is normal.
3. Open **Learn**. Wait for the live camera view, show your whole hand, and check that the app detects it. The first activation may take several minutes while dependencies download. The model is included and verified locally; it downloads only if the bundled copy is absent.
4. Return to Dashboard. Leave controller transmission stopped until pairing and RetroArch configuration are complete.

Checkpoint: Dashboard and Learn open from another device. A missing camera or model error is shown on Dashboard. If a page does not open, see Troubleshooting.

Stage 3 - Install the receiver on RetroPie

You must install the receiver on the Raspberry Pi as well as the application on the UNO Q. The recommended route downloads the same repository directly on RetroPie, so no manual file transfer from your computer is required.

1. Open a terminal on the Raspberry Pi running RetroPie. Use its keyboard and display, or an SSH connection if you have already enabled SSH. Use your actual RetroPie account; it is not necessarily named `pi`.
2. Run the commands below on the Raspberry Pi to install Git, download the source into your home folder, and run the receiver installer. Replace the UNO Q hostname before running the final command.
3. Read the report. Resolve any **FAIL** before pairing. **ACTION** means a step such as pairing or gameplay verification is still outstanding.

```
SH
sudo apt update
sudo apt install -y git
cd ~
git clone --branch dev https://github.com/mathan416/PowerGlove-Vision.git
cd PowerGlove-Vision
sudo python3 scripts/setup-machine.py retropie --peer UNO-Q-NAME.local
```

If you already have the checkout, open that folder instead of cloning again. The installer copies the required source into `/opt/powerglove-src`, installs commands into `/opt/powerglove/bin`, and creates missing settings under `/etc/powerglove/`. It also installs receiver dependencies, the delayed startup timer, RetroArch mapping, and game-launch hooks. Existing tokens, game mappings, and cabinet hooks are preserved. `--peer` sets the UNO Q address only when creating new launcher settings.

Checkpoint: `ls /opt/powerglove/bin/` lists `powerglove-pair`, `powerglove-receiver`, `powerglove-profile`, and `powerglove-retropie-hook`. The receiver waits for pairing if its token file is empty.

Stage 4 - Pair the two machines

Pairing gives both devices the same private token. Use the recommended one-time-code method after completing Stages 2 and 3.

1. On RetroPie, run `sudo /opt/powerglove/bin/powerglove-pair`. Leave the command running; it prints a 20-character code that expires after two minutes.
2. In your computer's browser, open `https://UNO-Q-NAME.local:8443/setup`. This is the secure Setup page; ordinary HTTP Setup cannot accept pairing credentials.
3. Enter your RetroPie hostname and its one-time code, then select **Prepare one-time code**.
4. Read the identifier after `ID` on the UNO Q matrix. Compare it with the beginning of the browser certificate's SHA-256 fingerprint. The locally generated certificate may cause a browser warning; verify the fingerprint before continuing.
5. If the identifiers match, select the confirmation checkbox, enter the six-digit PIN shown after `PN` on the matrix, and select **Complete pairing**.
6. On RetroPie, confirm that the helper reports completion and exits. Run `sudo systemctl status powerglove-receiver.service`; the receiver should be active.

If the code expires, restart the RetroPie command and prepare a new attempt. Never paste the private token into a document, screenshot, or support request.

Alternative: pair with your RetroPie password

Use this route only if RetroPie accepts SSH password login and your account can run `sudo` with that password.

1. Open secure Setup and enter the RetroPie hostname and username.
2. Select **Prepare password pairing** and compare the matrix `ID` with the browser certificate fingerprint.
3. If they match, select the confirmation checkbox, enter the matrix PIN and your RetroPie password, and complete pairing.
4. Confirm that the receiver service is active on RetroPie.

The password is used for one SSH operation and is not stored. If neither pairing route works, follow the token-management reference.

Stage 5 - Connect the virtual gamepad to RetroArch

1. On Dashboard, select a profile, wait for the camera, and show your hand. Use **Calibrate** if this is your first session or your resting position produces unwanted movement.
2. Select **Start controller**. This allows controller packets to reach RetroPie and creates the virtual input device.
3. On RetroPie, run `grep -A8 -B2 'PowerGlove Vision' /proc/bus/input/devices`. Look for the device name **PowerGlove Vision**. If it is missing, check pairing and the receiver service before changing emulator settings.
4. Use your physical controller to open RetroArch. Go to **Settings > Input > RetroPad Binds > Port 1 Controls** and select **PowerGlove Vision**. Menu labels can vary with the RetroArch version.

5. Check the D-pad, A, B, Start, and Select assignments. The installer provides an automatic mapping; adjust bindings only if needed, then save the controller profile or RetroArch configuration.
6. Test movement and buttons in a game. If your cabinet merges multiple controllers, also configure that merger to accept the virtual device.

Checkpoint: A gesture changes the intended control in the running game. Seeing the device name or a running service alone is not an end-to-end test. The extra Glove Zap signal is preserved for future native-glove integration; the standard gamepad path does not unlock Bad Street Brawler's native zap.

Stage 6 - Check automatic profiles and startup

1. Launch one of the registered NES or Famicom games. Check the active profile on Dashboard and its code on the UNO Q matrix.
2. If the game is not recognized, compare its complete filename with the entries in `/etc/powerglove/games.json` on RetroPie. Follow Register games and select profiles to add it.
3. End the game and confirm that **Gestures off** is selected. Launching an unregistered game or a different console system should also turn gestures off.
4. On RetroPie, run `systemctl is-enabled powerglove-receiver.timer` and `systemctl is-enabled powerglove-receiver.service`. Expect `enabled` for the timer and `disabled` for direct service startup. The timer starts the receiver 45 seconds after boot.
5. In App Lab, confirm that this copy of PowerGlove Vision is the default startup application. Keep only one copy set to start automatically.
6. Reboot each machine through its normal operating-system controls. After startup, confirm that Dashboard returns, the timer starts the receiver, and controller transmission remains stopped until you select **Start controller**.

The installer already adds the launch hooks. Do not add duplicate calls or overwrite existing cabinet hooks. Programs A, D, and H have no default game assignment; choose them on Dashboard or register a game explicitly.

For your first game, open the Gameplay Guide. For a compact reminder of connections and commands, use the Quick Reference.

Read the installation report

Result	What to do
PASS	Continue; the named check succeeded.
FAIL	Correct the reported problem and run the checks again.
ACTION	Complete the named user step, such as pairing or gameplay verification.

The installer returns exit code `0` when all checks pass, `1` for a failure, and `2` when user action remains. The current checks always request human gameplay verification, so a successful technical setup may still return `2`.

To rerun checks without installing or restarting anything, open the source directory on the named machine and use:

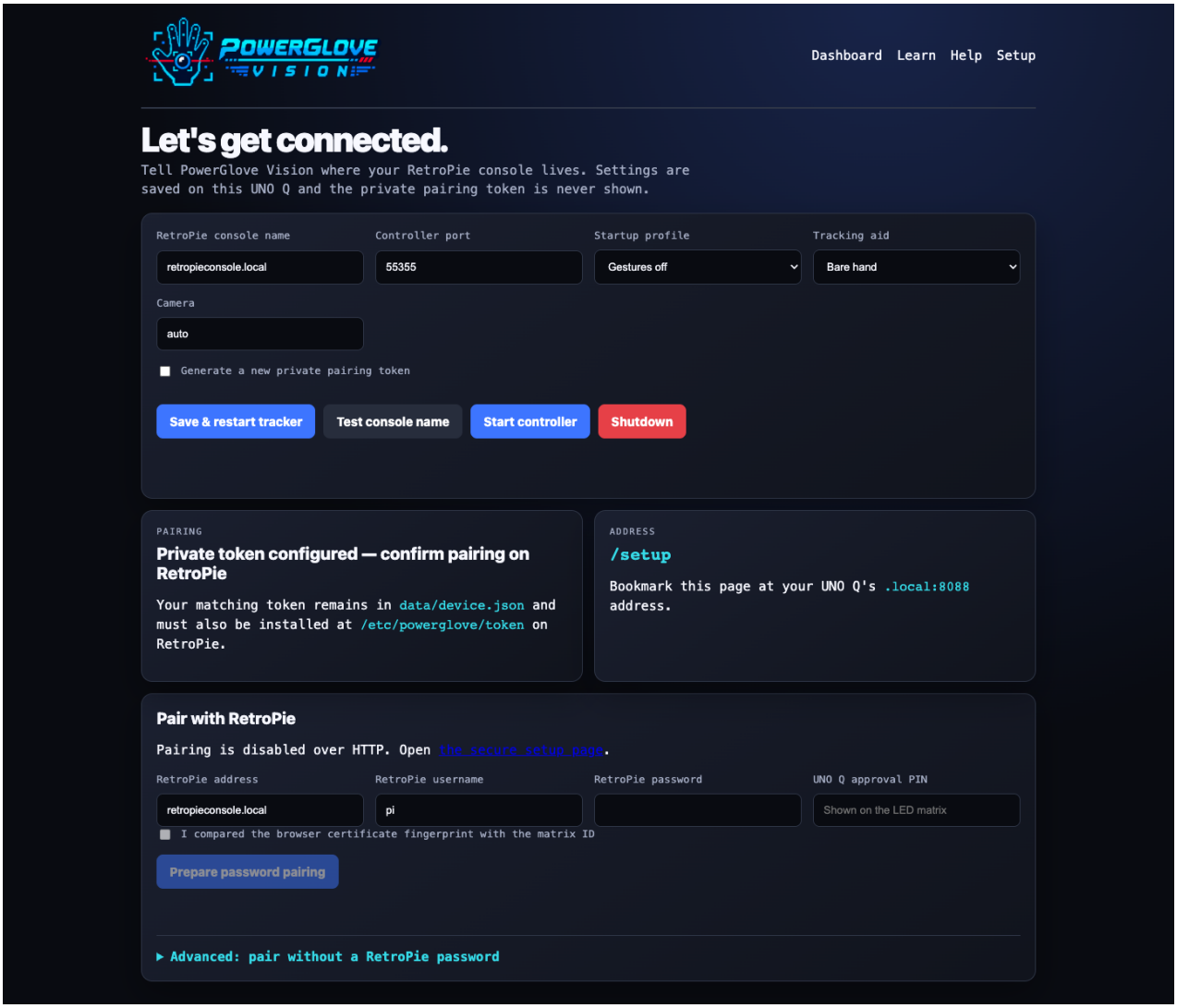
```
SH
# On RetroPie, from ~/PowerGlove-Vision:
sudo python3 scripts/setup-machine.py retropie --check
# On UNO Q, from /home/arduino/ArduinoApps/powerglove-vision:
sudo python3 scripts/setup-machine.py uno-q --check
```

Read the matrix and open the web pages

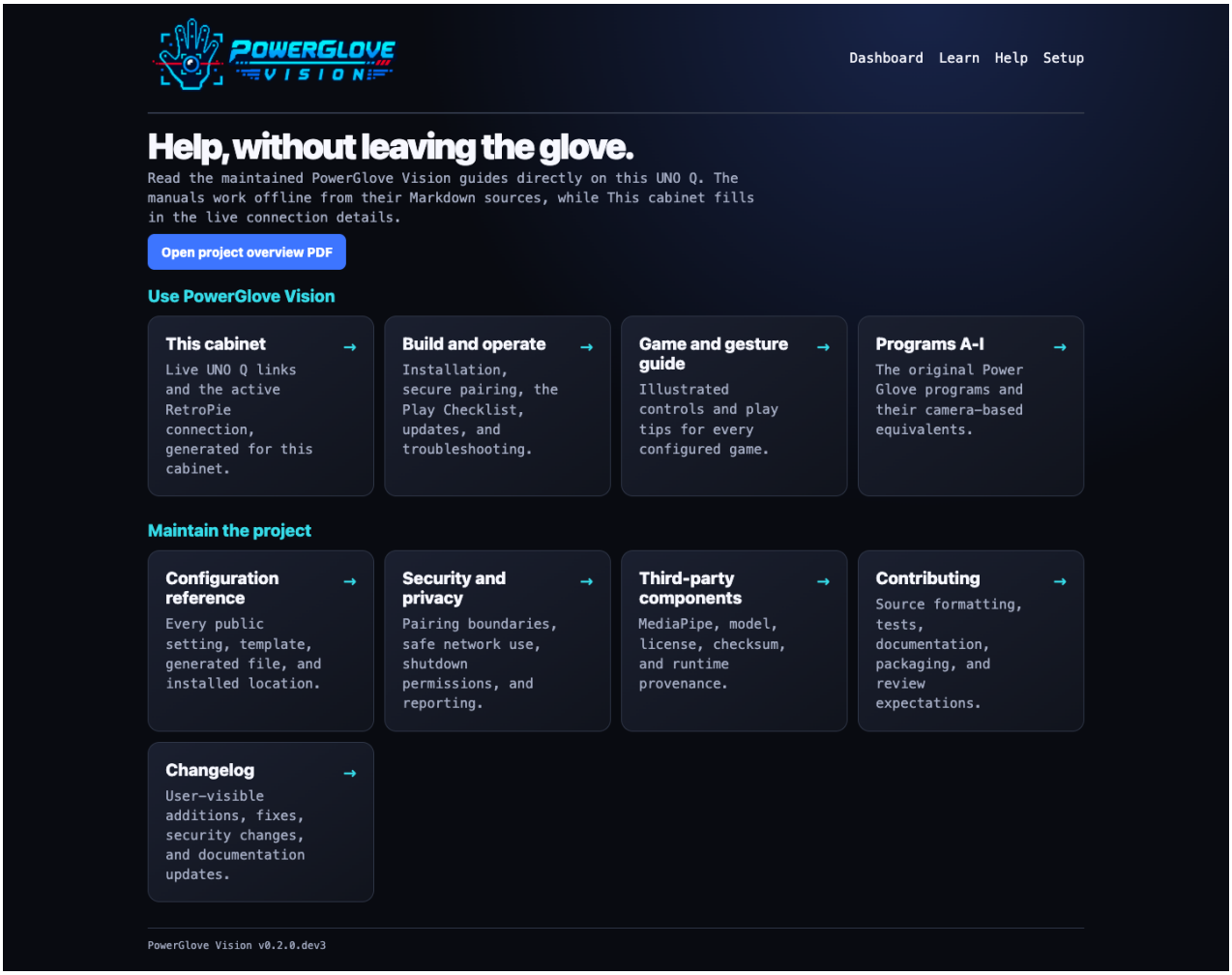
Display	Meaning
Animated hand	The app or vision runtime is loading.
Animated glove	Gestures are off.
Scanning `L`	Learn practice is active.
`T`	Tune gestures is active.
`A`-`I`, `BS`, or `GB`	The corresponding profile is active.
Pulsing profile code	A calibrated hand is being tracked.
Blinking X	The camera or vision runtime needs attention.

An animation does not prove that Linux is running or that shutdown has finished.

Page	Address
Dashboard	`http://UNO-Q-NAME.local:8088/debug`
Learn	`http://UNO-Q-NAME.local:8088/learn`
Games (lower Setup section)	`http://UNO-Q-NAME.local:8088/setup#games-section`
Help and printable manuals	`http://UNO-Q-NAME.local:8088/help`
Connection settings	`http://UNO-Q-NAME.local:8088/setup`
Secure pairing	`https://UNO-Q-NAME.local:8443/setup`



Setup page; use HTTPS to enable pairing



Help page with links to the local manuals

Help serves the public manuals, illustrations, and PDFs locally. **This cabinet** shows addresses derived from your current browser connection and public device settings. It never displays the token. The standalone Quick Reference is excluded from the public package; the live cabinet page supplies local details.

Play Checklist

1. Power the RetroPie and UNO Q; leave the camera connected to the powered hub.
2. Open `http://UNO-Q-NAME.local:8088/debug`.
3. Select the active profile on the Dashboard, then confirm the expected profile and a detected hand. The saved startup profile remains on Setup.
4. On first use, or after changing your camera or playing position, select **Calibrate** while holding a comfortable neutral pose. Otherwise reuse the saved calibration.
5. Select **Start controller** only when you are ready to play.
6. Launch the game and confirm its profile code on the matrix.
7. Select **Stop controller** before adjusting the camera or leaving the cabinet.

8. Read the shutdown limitation before disconnecting power. **Shutdown** requests a graceful halt, but the tested board restarts; an offline website is not proof that it is safe to unplug.

PowerGlove Vision deliberately boots with controller delivery stopped. Vision and the dashboard keep running so setup never generates surprise game inputs. **Shutdown** is different: it halts Linux on the UNO Q. The tested board automatically restarts; remaining halted is not guaranteed.

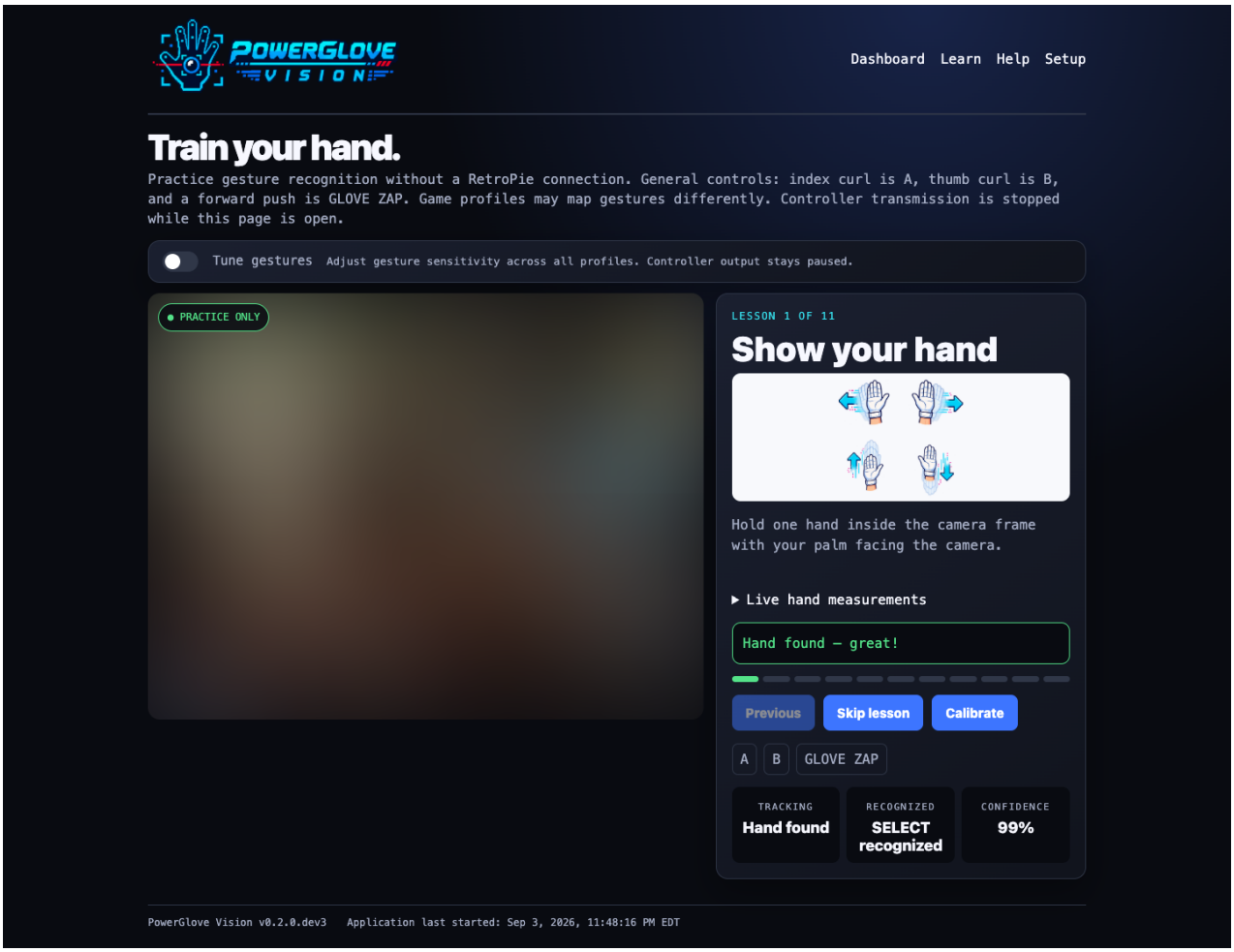
Learn before you launch

Dashboard and Learn show **Starting camera and gesture tracking** with elapsed seconds during initialization. The first activation can take longer while the camera and tracker load. Wait for vision to become active before centring; the centring button is disabled during startup. Gestures off keeps the camera off rather than briefly activating it at boot.

Open `http://UNO-Q-NAME.local:8088/learn`. Learn mode automatically stops controller transmission, starts the camera when necessary, and guides you through eleven exercises with live feedback on the gestures it recognizes. RetroPie does not need to be online. Leaving Learn restores the selected profile and its camera state; loading or refreshing the Dashboard also reapplies the selected mode.

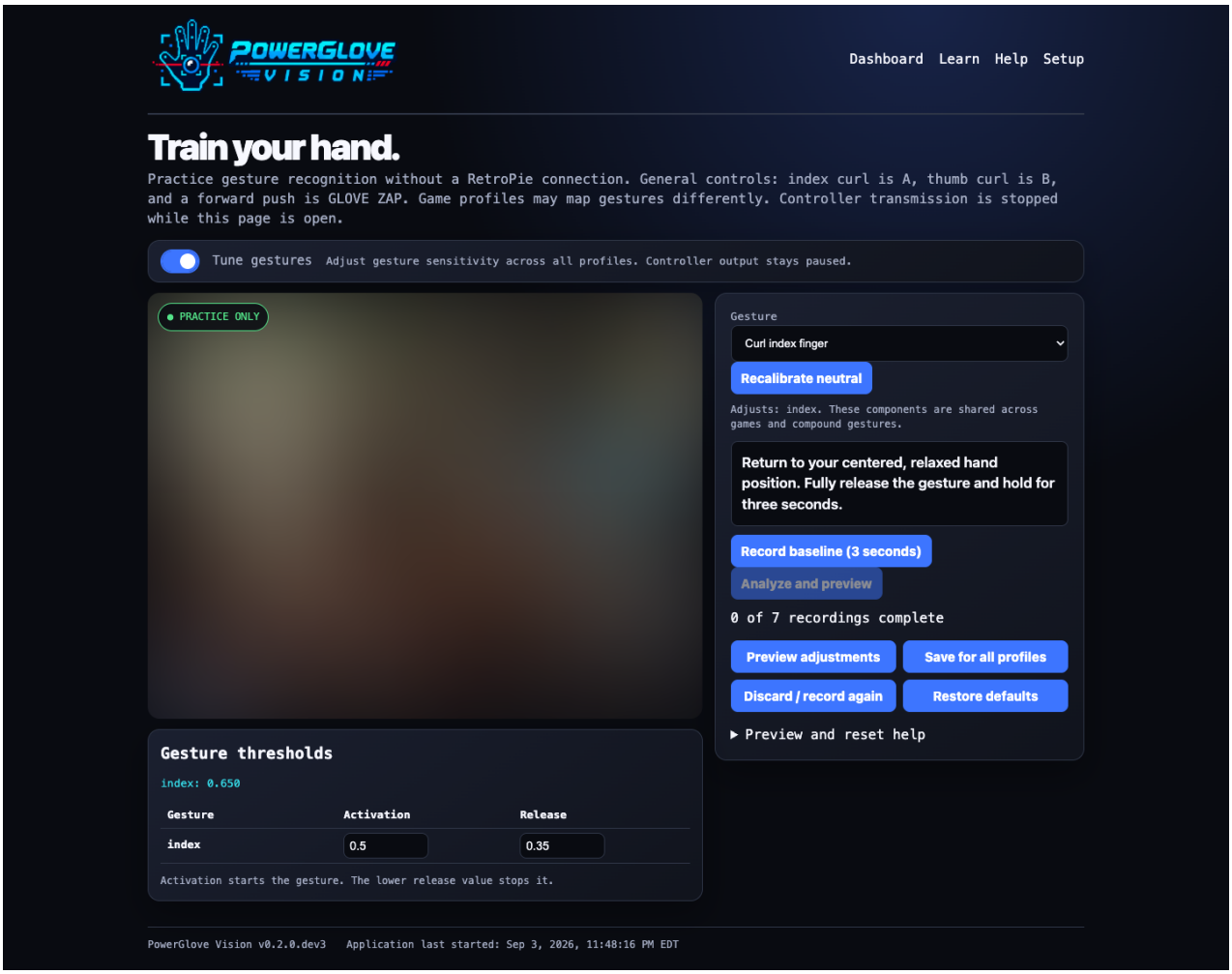
Switch on **Tune gestures** to record a baseline and three gesture-and-release repetitions, preview suggested thresholds, and save them across all profiles. See Tune gesture sensitivity for the numbered walkthrough. Tuning keeps controller delivery stopped; explicitly start it again from Dashboard when ready to play.

Open **Setup -> Games** to edit the installed RetroPie game mappings. Validate before saving, then restart the game. **Download backup** and **Restore previous save** provide recovery options. Both devices need the current software; RetroPie setup installs the paired Games service on TCP `55358`.



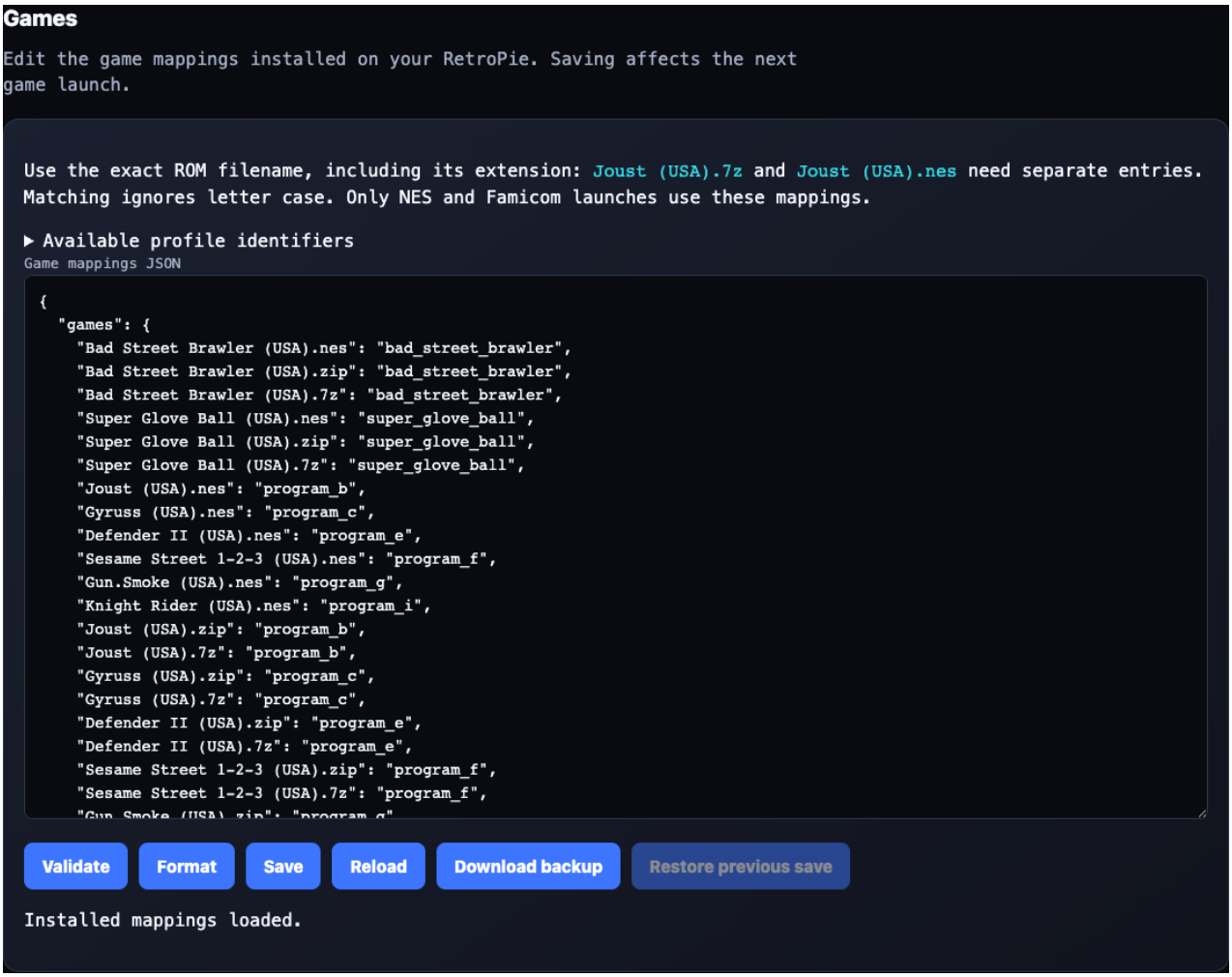
PowerGlove Vision Learn page

Camera imagery in these screenshots is blurred for privacy.



Tune mode, with thresholds below the camera and recording controls alongside

In Tune mode the matrix shows **T**; ordinary Learn practice shows **L**. The Activation and Release table sits below the camera. Use the instructions and recording buttons beside it to work through the seven recordings.



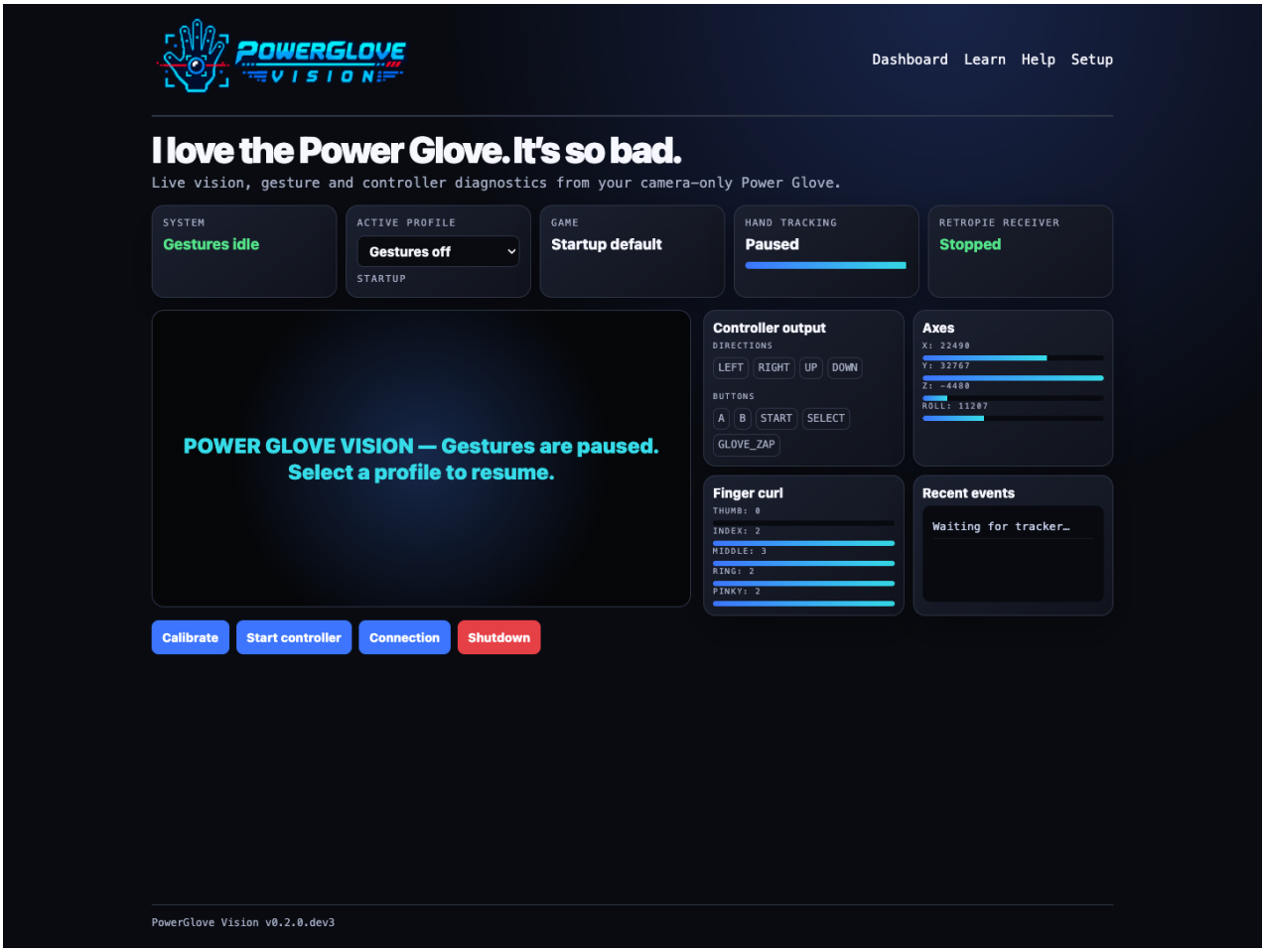
Games editor in the lower part of Setup

Scroll below pairing to find **Games**. Changes take effect on the next game launch.

For a white glove, use a darker background. For a black glove, use a lighter background. Even lighting, a simple scene, and keeping the whole hand in frame matter more than glove colour.

Read the live dashboard

Debug shows the camera overlay, active profile, tracking confidence, generated D-pad and button readings, analogue axes, finger curl, and recent gesture events. **Right** or **Left** in the camera overlay identifies the detected hand, not a movement direction. Its confidence score describes that handedness classification, not confidence in a movement command. Use the D-pad and button readings and axes to inspect actual generated controls. Its profile selector changes the current session immediately. Selecting **Gestures off** releases controls, closes the camera and shows a friendly idle panel; selecting another profile starts vision using the saved calibration. Recalibrate after moving the camera or changing your playing position, or if your resting hand produces unwanted movement.



PowerGlove Vision Debug dashboard

Workshop - Updates and maintenance

UNO Q updates over Wi-Fi

Use this route after the initial App Lab installation. It updates the Linux application; matrix firmware changes still need **Run** in App Lab.

Set up SSH key access once

1. On your development computer, check for an existing public key in `~/.ssh/`. Use only a file ending in `.pub`; never copy its matching private key.
2. If you do not have a key, run `ssh-keygen -t ed25519`. Accept the suggested location only if it does not replace an existing key, and follow the passphrase prompts.
3. Open your public-key file and copy its complete single line. For the default key, run `cat ~/.ssh/id_ed25519.pub`.
4. Connect with `ssh arduino@UNO-Q-NAME.local`. On the UNO Q, run `install -d -m 0700 ~/.ssh`, then `nano ~/.ssh/authorized_keys`.
5. Add the public key on a new line, preserving any existing keys. Save with Ctrl+O, confirm the name, and exit with Ctrl+X.

6. Run `chmod 0600 ~/.ssh/authorized_keys`, then `exit` to return to your computer. If your private key has a passphrase, make it available through your computer's SSH agent before the unattended deployment check.
7. Run `ssh -o BatchMode=yes arduino@UNO-Q-NAME.local hostname`. Continue only when it prints the UNO Q hostname without requesting a login password.

Update the application

1. On your development computer, open your project checkout and review local changes with `git status --short`.
2. If you are updating from GitHub, run `git pull --ff-only`. Resolve any reported local-change or branch conflict before deploying. Keep the version compatible with the RetroPie installation.
3. Run the deployment command below. It preserves private `data/`, restarts the application, and checks its web pages.
4. Open Dashboard and Learn to confirm the updated app works. If you changed the matrix sketch, also rebuild and run it through App Lab.

```
SH
scripts/deploy-uno-q-wifi.sh arduino@UNO-Q-NAME.local
```

Use the board's current IP address if its local name is temporarily unavailable. If SSH access fails, reconnect through App Lab before retrying deployment. For local-name failures, see hostname resolution.

Repair or update the shutdown helper

The helper requests `systemctl --no-block halt`, not `poweroff`: the UNO Q can reboot after a power-off request. A halt requests that Linux stop; it does not disconnect electrical power. LEDs may remain lit. **Known limitation, confirmed September 3, 2026:** our UNO Q restarted after reaching the halt target both through a powered USB-C hub and when connected directly to a Mac. Shutdown is therefore not a verified way to keep this board stopped. Do not treat the website disappearing as a safe-to-unplug indicator or rely on a fixed countdown. See Arduino shutdown guidance.

The App Lab container is intentionally unprivileged and cannot halt the Linux host. The machine installer already installs the fixed-purpose systemd path helper. To repair or update it separately, run this command from your development computer's project checkout:

```
SH
scripts/install-uno-q-shutdown-helper.sh arduino@UNO-Q-NAME.local
```

Enter the UNO Q `arduino` account password at the remote `sudo` prompt. The script does not read or store it. The helper watches only the fixed `data/shutdown-request` path and can perform only a system halt. After it is installed, **Shutdown** is available on Dashboard and Setup. Each press requires browser confirmation and warns that the UNO Q may restart automatically and that a disconnected website does not confirm it is safe to remove power. Its boot-time tmpfiles rule recreates the readiness marker if the UNO Q reboots or App Lab replaces the application directory.

Verify the helper without triggering shutdown:

```
SH
ssh arduino@UNO-Q-NAME.local
systemctl is-enabled powerglove-system-shutdown.path
systemctl is-active powerglove-system-shutdown.path
exit
```

Expected results are `enabled` and `active`. Checks on the cabinet on September 3, 2026 confirmed that the watcher was enabled and active, the readiness marker existed, and both pages displayed the Shutdown button. Requests without confirmation were rejected. Do not test the accepted API path unless you intend to shut down the UNO Q.

RetroPie updates

1. On RetroPie, back up customized files under `/etc/powerglove/`, especially `games.json` and `launcher.json`, using your normal private backup method.
2. Open the original source checkout, normally `~/PowerGlove-Vision`. The installed copy under `/opt/powerglove-src` is not a Git checkout.
3. Run the commands below. Review `git status --short` before pulling; if Git reports a conflict, resolve it before running the installer.
4. Resolve any **FAIL** in the installer report, then launch a registered game and check its profile and controls. The installer preserves existing settings and tokens.

```
SH
cd ~/PowerGlove-Vision
git status --short
git pull --ff-only
sudo python3 scripts/setup-machine.py retropie --peer UNO-Q-NAME.local
```

Duplicate App Lab entries

Importing a newer ZIP may create a timestamped copy. The supported host installer and shutdown helper require `/home/arduino/ArduinoApps/powerglove-vision`. Keep the working application at that path; do not run host setup from a duplicate. Use the Wi-Fi update procedure for routine Linux application changes. Before removing any duplicate in App Lab, confirm which copy has your private settings and keep only the intended application set to start at boot.

Troubleshooting

Matrix shows a blinking X

- Confirm that an active gesture profile is selected. **Gestures off** should display the animated glove attract sequence, never the error X.
- Confirm the camera is connected through the powered hub.
- Try another hub port or USB cable.
- Check whether Linux sees a USB camera; internal `qcom-venus-encoder` and `qcom-venus-decoder` nodes are codecs, not your camera.
- Restart the app after checking power and cabling.

Camera appears only after reconnecting it

This usually indicates USB enumeration or power trouble. Keep the powered hub energized before starting the UNO Q, try another cable, and avoid passive adapters. The app itself waits for a camera and should recover when it appears.

First start takes several minutes

A slow first start is expected because the UNO Q must download its private Python 3.12 runtime and vision libraries. Later launches reuse the persistent cache. Keep internet access available and watch the App Lab log for progress.

Setup page does not open

- Ordinary settings: `http://UNO-Q-NAME.local:8088/setup`
- Secure pairing: `https://UNO-Q-NAME.local:8443/setup`
- Try the board's IP address if `.local` does not resolve.
- HTTPS and HTTP are not interchangeable on these ports.

Password pairing fails

- Prepare a new attempt and use its new matrix PIN.
- Confirm the RetroPie username and password can log in through SSH.
- The account must be allowed to run `sudo` with that password.
- Prefer the one-time-code method if password SSH is disabled.

Controller does not appear on RetroPie

```
SH
systemctl is-enabled powerglove-receiver.service
systemctl is-enabled powerglove-receiver.timer
sudo systemctl status powerglove-receiver.service
sudo systemctl status powerglove-receiver.timer
sudo journalctl -u powerglove-receiver.service -n 100 --no-pager
ls -l /dev/uinput
```

Allow 45 seconds after boot. Confirm `uinput` is loaded and `/etc/powerglove/token` is not empty. Expected boot enablement is `disabled` for the service and `enabled` for the timer. The virtual controller appears only after an authenticated packet arrives.

Frontend slowdown or USB-device conflicts

Confirm the receiver service was not enabled directly. Stop it, restart EmulationStation, and start the receiver afterward as an A/B test. If the frontend becomes responsive, restore the supplied timer. Also ensure Pixelcade's `game-select` and `system-select` directories contain only one executable hook each if you use a BitPixel display; executable backup scripts are additional hooks.

Controller exists but does not move

- Select **Start controller** on Setup or Debug.
- Confirm a calibrated hand and controller output on Debug.
- Verify the receiver address and UDP 55355 connectivity.
- Check whether an existing cabinet input merger filters the virtual device.

Profiles do not change

A **profile queued** message confirms authentication and queue admission. Wait for Dashboard to show the new profile; camera startup may still be in progress. For timeouts, check that the UNO Q publishes UDP `55356` through its profile relay. Follow Check a queued profile change for the command and recovery steps. Check the exact ROM filename, including its archive extension, if the selected profile is **off**.

- Test `powerglove-profile` manually.
- Check `uno_q` and `token_file` in `/etc/powerglove/launcher.json`.
- Confirm both runcommand hooks call the supplied helper scripts.
- Match the exact ROM basename in `/etc/powerglove/games.json`.

FAQ: What if the console name cannot be resolved?

1. In **Connection**, enter your console's actual hostname, such as `RETROPIE-NAME.local`, then select **Test console name**. Use a hostname or IPv4 address, not `http://`, a port, or a page path. This tests resolution from the UNO Q app; successful lookup on your laptop alone is not sufficient.
2. Confirm the RetroPie console is powered on and connected to your LAN. On its terminal, run `hostname` and `hostname -I` to confirm its name and current addresses. Do not assume an old DHCP address is still correct.
3. From the PowerGlove source directory on RetroPie, run `sudo python3 scripts/setup-machine.py retropie --check`. Check Avahi with `systemctl is-active avahi-daemon` and `systemctl is-enabled avahi-daemon`. If setup is incomplete, rerun `sudo python3 scripts/setup-machine.py retropie --peer UNO-Q-NAME.local`, using your board's actual name, and review every FAIL or ACTION result.
4. On the UNO Q, from the app directory, run `sudo python3 scripts/setup-machine.py uno-q --check`. This checks the configured destination from inside the application. If installation is incomplete, rerun `sudo python3 scripts/setup-machine.py uno-q`. Do not manually patch `.cache/app-compose.yaml`: App Lab regenerates it. The shipped resolver brick supplies the persistent configuration.
5. Check that both machines are on a network that allows communication between devices. Guest Wi-Fi, client isolation, VPN routing, separate VLANs, or multicast filtering can prevent `.local` discovery. mDNS uses UDP port 5353; do not disable your firewall wholesale or expose the app to the Internet to fix discovery.
6. As a diagnostic or fallback, enter RetroPie's current LAN IPv4 address in **Connection** and test again. If that works while the name fails, investigate mDNS. For continued use, reserve that address in your router so DHCP does not change it. Save the intended destination using the normal Connection workflow; changing the address does not replace pairing credentials. If RetroPie also contacts the UNO Q by name, check that reverse direction separately.

7. If neither name nor IP works, investigate connectivity and the service itself, not just Avahi. A successful name test only establishes name resolution; pairing, the receiver, controller output, and emulator mappings must also work. Retry after boot has finished, then collect the exact error and setup-check results if it still fails. Never share tokens, passwords, or private SSH keys.

After fixing the problem, reboot both machines and repeat **Test console name** before testing gameplay. The app-owned resolver has been verified across a UNO Q reboot and a changed RetroPie DHCP address; no fixed IP entry is required for `.local` use.

Uninstalling

On RetroPie:

```
SH
sudo systemctl disable --now powerglove-receiver.timer
sudo systemctl disable --now powerglove-receiver.service
sudo rm /etc/systemd/system/powerglove-receiver.timer \
  /etc/systemd/system/powerglove-receiver.service
sudo systemctl daemon-reload
```

Remove only the PowerGlove lines from the runcommand hooks. After backing up custom profiles, `/opt/powerglove`, `/opt/powerglove-src`, and `/etc/powerglove` may be removed manually.

On the UNO Q, stop the app, disable **Run at startup**, and remove it through Arduino App Lab. Its private `data` directory contains the device token and cached runtime.

Remove the host shutdown helper separately:

```
SH
sudo systemctl disable --now powerglove-system-shutdown.path
sudo rm /etc/systemd/system/powerglove-system-shutdown.path \
  /etc/systemd/system/powerglove-system-shutdown.service \
  /etc/tmpfiles.d/powerglove-system-shutdown.conf
rm -f /home/arduino/ArduinoApps/powerglove-vision/data/.shutdown-enabled
sudo systemctl daemon-reload
```

Project note

PowerGlove Vision is an independent MIT-licensed hobbyist project by Iain Bennett. Nintendo, NES, Power Glove, Bad Street Brawler, Super Glove Ball, and other marks belong to their respective owners. No ROM images are distributed with this project. Third-party runtime components retain their own licenses and terms as documented in THIRD_PARTY_COMPONENTS.md.